pfi

Python for Fixed Income

COURSE 9 · DEPLOYMENT

Request Logs and Monitoring

WEEK 3 · DAY 2

Created by **Jeff Lenamon**

This course is educational content created in a personal capacity. Nothing here is investment advice or a recommendation to buy or sell any security. All portfolio data is synthetic.

Today: request logs and monitoring

Today

1. What to log, and what never to log
2. log_middleware on the bond service
3. On a platform, standard output
4. A month of requests by path
5. Percentiles, and the index that differs
6. Alerts by rule
7. An outage, from the record
8. What Vercel kept of the course's requests

Two logs

Our own

10 requests, through the test client

on a fake clock: the same every run

A month, synthetic

19,664 requests, 2026-09-03 to 2026-10-02

three features planted for today

A deployed service's only record of what it did is what it wrote down at the time.

What a log's numbers are called

Requests, latency in milliseconds, an error rate: about the service, never about a bond.

Allowed names

requests

p95 latency in ms

error rate

Never called

performance of the desk

P&L

a health score of a market

A request that took 12 ms says nothing about the bond it asked about, or the market.

What to log, and what never to log

Seven columns

time_utc: UTC, with milliseconds

request_id: 16 hexadecimal characters

path: without the query string

status: 200, 404, 422, 500

latency_ms

bytes_out

refused: the guardrail, counted

Never

a request's body: the bond, the question

a query value

a header's value

the client's address

a response body (a 422 echoes the body)

The stated exception: day 13's model keeps a separate log of its inputs, for drift, and nothing about the sender.

How a middleware works

The problem	A log has to be written for every request, and an endpoint that has to remember to write it will forget one day
The method	One function FastAPI runs around every request: read the clock, hand the request on, take the response, read the clock again
The line	Path, status, the response's size, the elapsed ms, as one JSON line, handed to a sink: any function that takes a line
You control	The sink (a file append, print, a logger's info method) and, in a notebook, the clock
What comes out	The same app with the middleware attached, and one line per request wherever the sink puts it

log_middleware on the bond service

```
wall, perf = deploykit.fake_clock("2026-10-02T14:00:00", 12.0)
app = deploykit.log_middleware(deploykit.bond_app(), write_line, clock=perf, wall=wall)
```

Ten requests through the test client: [200, 200, 200, 200, 200, 200, 422, 404, 200, 200]

```
{"time_utc": "2026-10-02T14:00:00.000Z", "request_id": "39bcadea4c81a848", "path": "/health",
  "status": 200, "latency_ms": 12.0, "bytes_out": 44, "refused": false}
```

7

columns in every line, and nothing else

no

client's name or yield in a line

2

sinks: a file here, print on a platform

The client's name in the line: no; the yield 5.06 in the line: no.

Every setting of logging, read from the installed Python

Object	Setting	Default	What it does	The course sets
Formatter	fmt	None	the pattern a record is written with; None gives the message a	"%(message)s": the middleware's line, which
Formatter	datefmt	None	the time's format, when the pattern asks for %(asctime)s	
Formatter	style	'%'	the pattern's syntax: % (the default), { for str.format, $ for	
Formatter	validate	True	check the pattern against the style when the formatter is made	
Formatter	defaults	None	values for pattern fields a record lacks	
FileHandler	file name	required	the file the lines go to	requests_logging.jsonl in the work folder
FileHandler	mode	'a'	how the file is opened: a (append) by default	"w": a fresh file each run
FileHandler	encoding	None	the file's text encoding; None is the platform's	"utf-8"
FileHandler	delay	False	open the file at the first line, not when the handler is made	
FileHandler	errors	None	what to do with a character the encoding cannot write	
StreamHandler	stream	None	the stream written to; None is standard error	
basicConfig	file name	not set (keyword)	a FileHandler on this file instead of a StreamHandler	
basicConfig	format	not set (keyword)	the root handler's Formatter pattern	
basicConfig	level	not set (keyword)	the root logger's level	
basicConfig	stream	not set (keyword)	a StreamHandler on this stream (standard error when nothing is	

22 settings, 4 of basicConfig's 11 keywords shown; the course sets four. StreamHandler writes to standard error by default: a platform sink passes sys.stdout, or stays with print.

On a platform, the log goes to standard output

A function's disk is not a log store

1. Read-only, apart from /tmp scratch space in one running copy
2. Copies come and go: archived when idle, many at once under load
3. What a function prints is kept, grouped by request

Vercel Functions: a read-only file system, writable /tmp up to 500 MB (vercel.com/docs/functions/runtimes, read 2026-10-12).

Vercel Hobby: runtime logs kept for 1 hour; Pro 1 day (vercel.com/docs/logs/runtime, read 2026-10-12).

Vercel runtime logs: up to 256 lines and 1 MB a request, 256 KB a line (vercel.com/docs/logs/runtime, read 2026-10-12).

Locally a file, on a platform standard output: the same log_middleware, two sinks.

How log_summary works

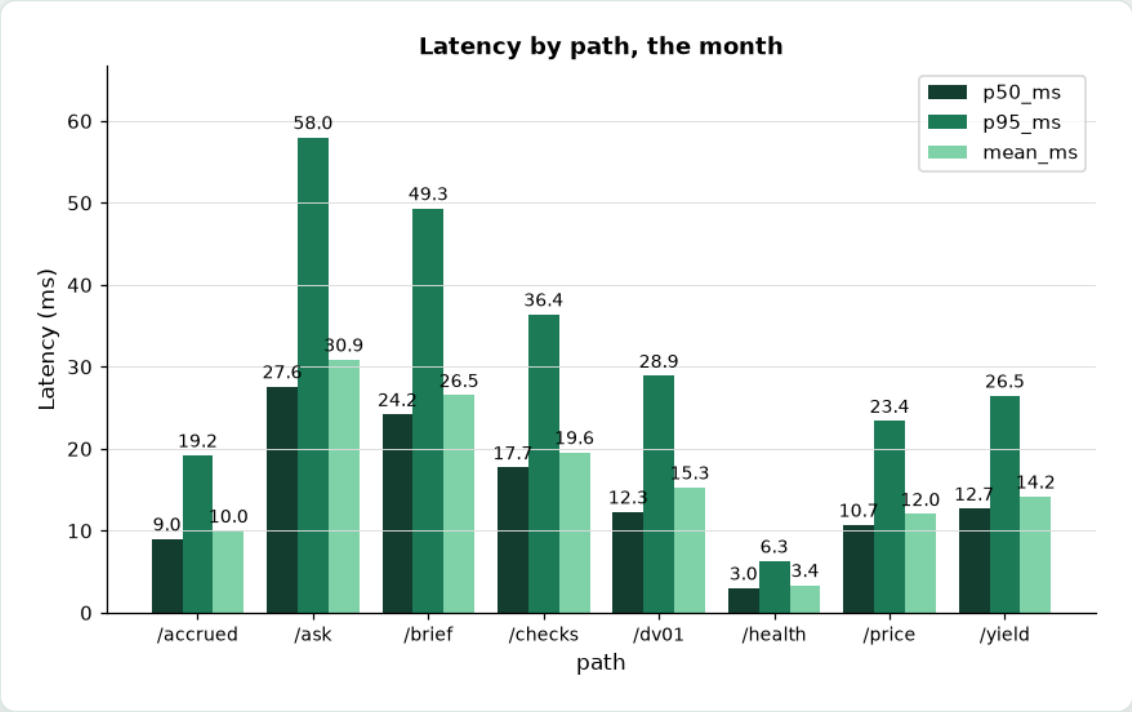
The problem	A month of request lines is too long to read: it has to be read as one row per endpoint
read_log	Every line into one typed table: a UTC timestamp, integer statuses and sizes, a float latency, refused as True or False
log_summary	Group by path; count the rows, the 4xx, the 5xx, the refused; the latency's median, p95 (linear), and mean
You control	The column to group by: path by default, a day or an hour works too
What comes out	One row per group, seven columns, read column by column

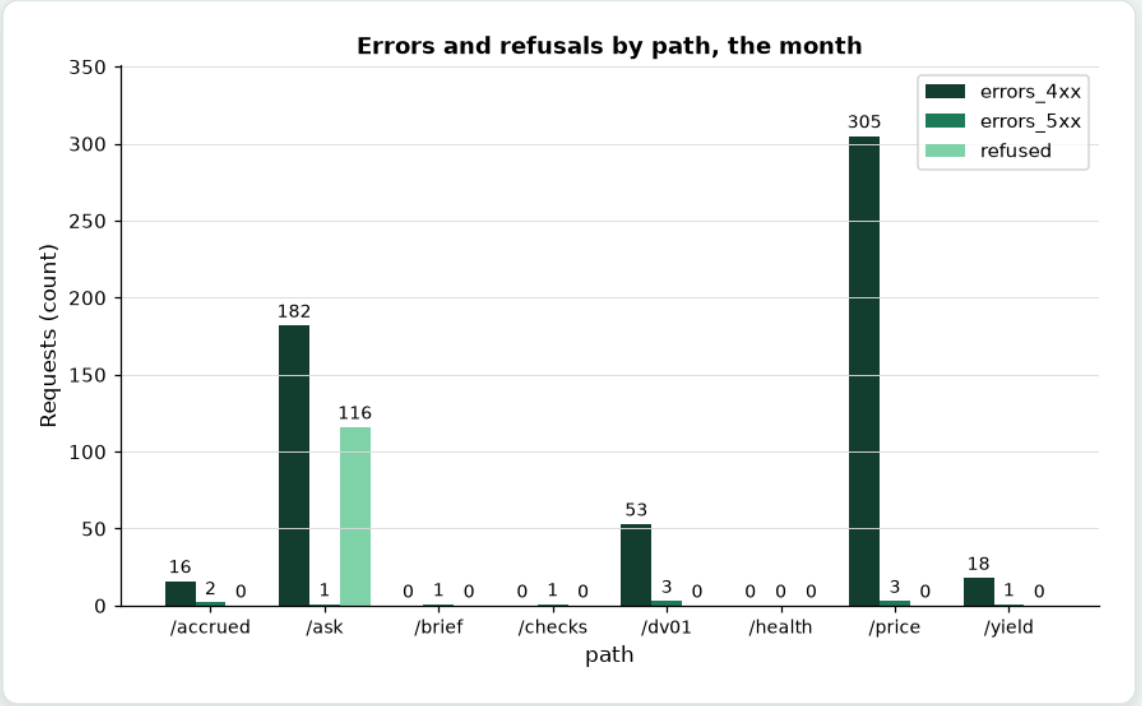
A month of requests, by path

path	requests	errors_4xx	errors_5xx	refused	p50_ms	p95_ms	mean_ms
/accrued	1338	16	2	0	9.0	19.2	10.0
/ask	1370	182	1	116	27.6	58.0	30.9
/brief	1915	0	1	0	24.2	49.3	26.5
/checks	1003	0	1	0	17.7	36.4	19.6
/dv01	4625	53	3	0	12.3	28.9	15.3
/health	2935	0	0	0	3.0	6.3	3.4
/price	4971	305	3	0	10.7	23.4	12.0
/yield	1507	18	1	0	12.7	26.5	14.2

19,664 requests. Percentiles, not the mean: a few slow requests pull a mean up and say nothing about the typical one.

The month by path, drawn





The p95 sits well above the median on every path; the refusals are all on /ask.

How a percentile works

The problem	A mean is pulled up by a few slow requests: a service is watched by the latency most requests stay under
The method	Sort; position 0.95 x (n - 1) usually falls between two values; linear interpolation takes the point that far between them
Other rules	PERCENTILE.EXC counts from 1 over n + 1; an index into the sorted list takes a whole value and never interpolates
You control	The percentile and the method: the setting people forget to name
What comes out	One number in ms, equal between two tools only when both use the same rule

Percentiles: name the method

19.05 ms

linear: NumPy, pandas, PERCENTILE.INC

19.95 ms

PERCENTILE.EXC's rule

20.00 ms

by index: sorted(x)[int(0.95 * n)]

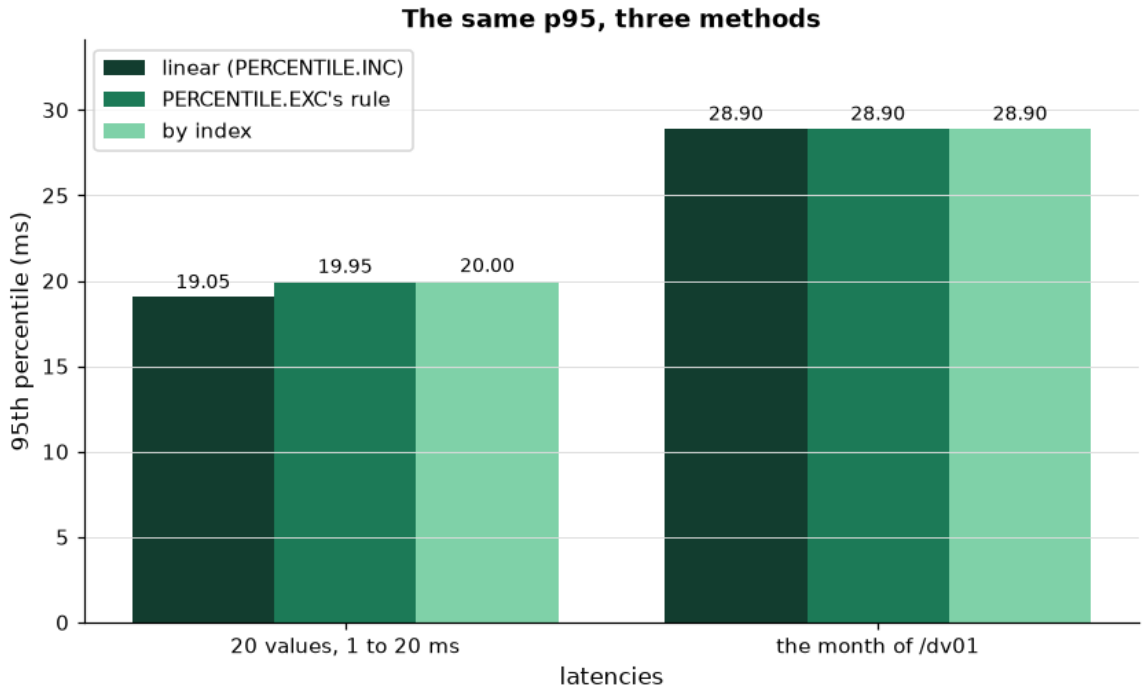
The p95 of 20 hand-made latencies, 1 ms to 20 ms: three methods, three numbers.

The month's /dv01 requests

All three give 28.90 ms: the latencies repeat at 0.1 ms, so the methods land on equal values.

The course uses linear everywhere, and log_summary's docstring says so.

Three methods, one p95



Latencies	Linear	EXC rule	Index
20 values	19.05	19.95	20.00
month, /dv01	28.90	28.90	28.90

On a small sample, name the method; on a month of traffic the bars are the same height.

How health_report works

The problem	A month's summary averages a bad day away, and a reader finds what they expect: the rule is written before the data is seen
The method	Group the log by UTC day; for each day the requests, the p95 (linear), and the error share: requests with status 400 or above
The verdict	alert when the p95 is over its limit or the error share over its limit; ok otherwise
You control	The two limits, each a number in a file a reviewer can diff
What comes out	A dictionary, one entry per day with four figures and a status, for a daily job to print or a notebook to draw

Alerts by rule

```
days: 30; alerts: 2
2026-09-17: 605 requests, p95 101.2 ms, error share 2.1%
2026-09-24: 851 requests, p95 32.5 ms, error share 29.8%
on the other days: p95 at most 37.7 ms, error share at most 2.9%
```

154.0 ms

/dv01 p95 on the slow day

61.8%

/price error share, burst day

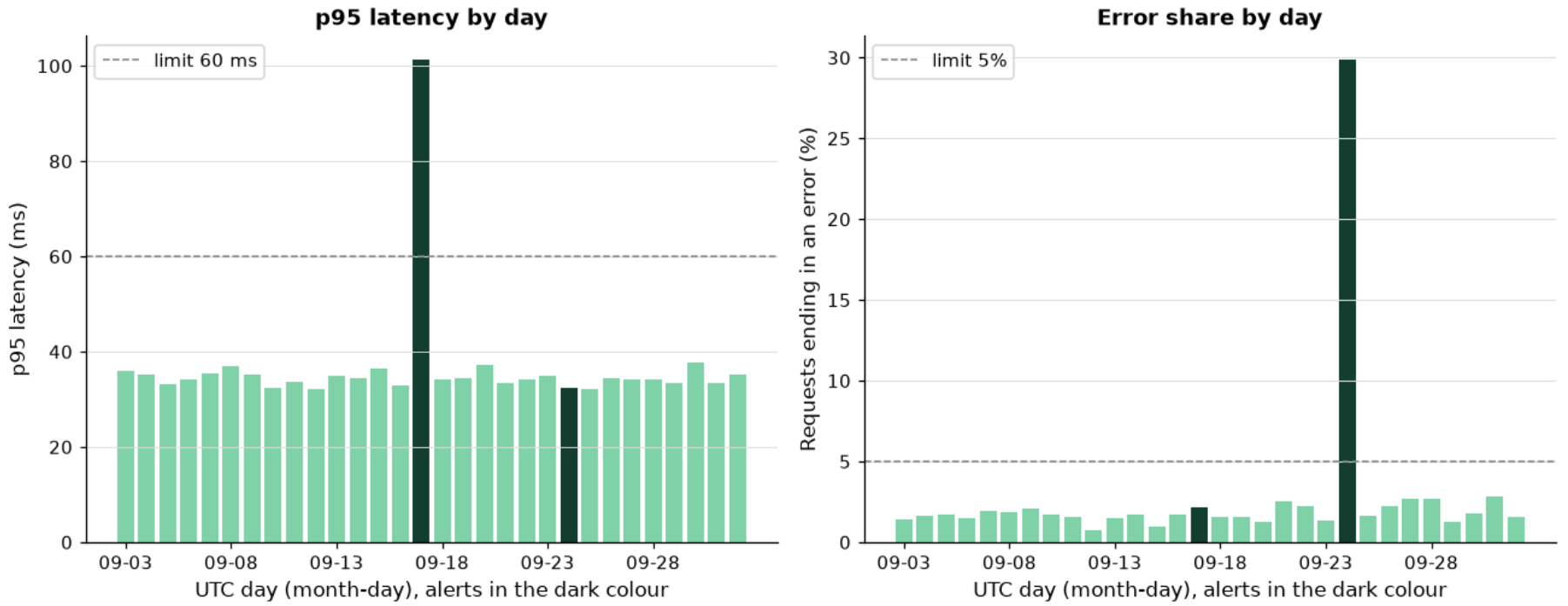
14:00

the hour of the 422 burst, UTC

422s on /price on 2026-09-24, by UTC hour: {13: 1, 14: 240, 19: 1}

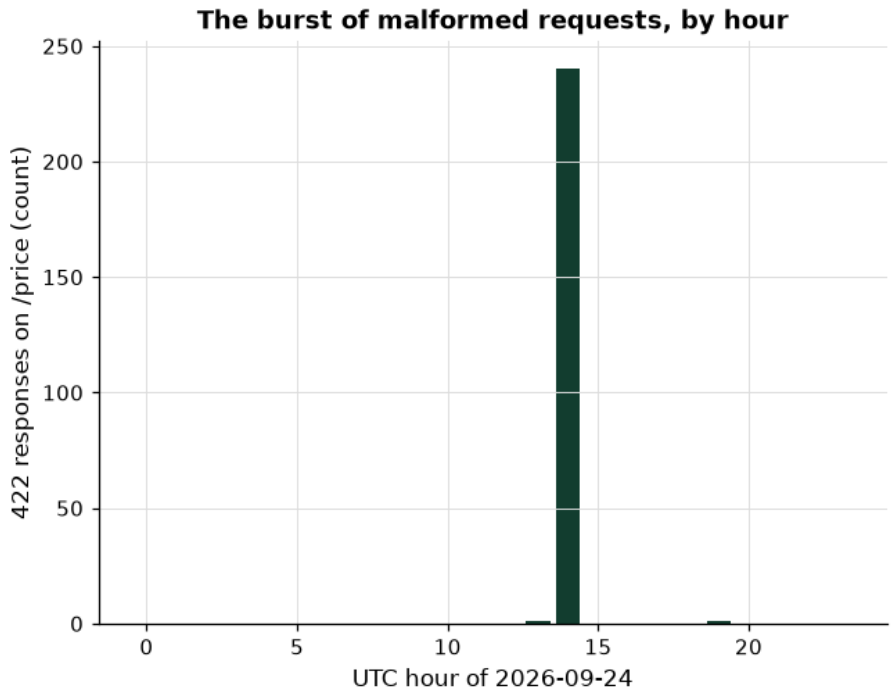
A status page is the same rule, published for a service's users.

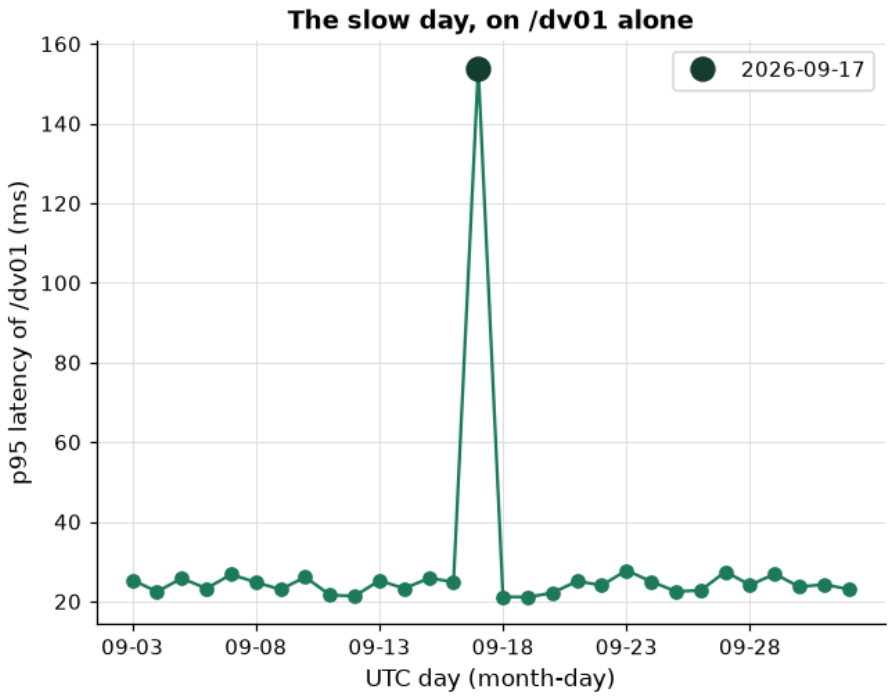
The month, day by day



The alert days: 2026-09-17 (p95 101.2 ms, errors 2.1%); 2026-09-24 (p95 32.5 ms, errors 29.8%).

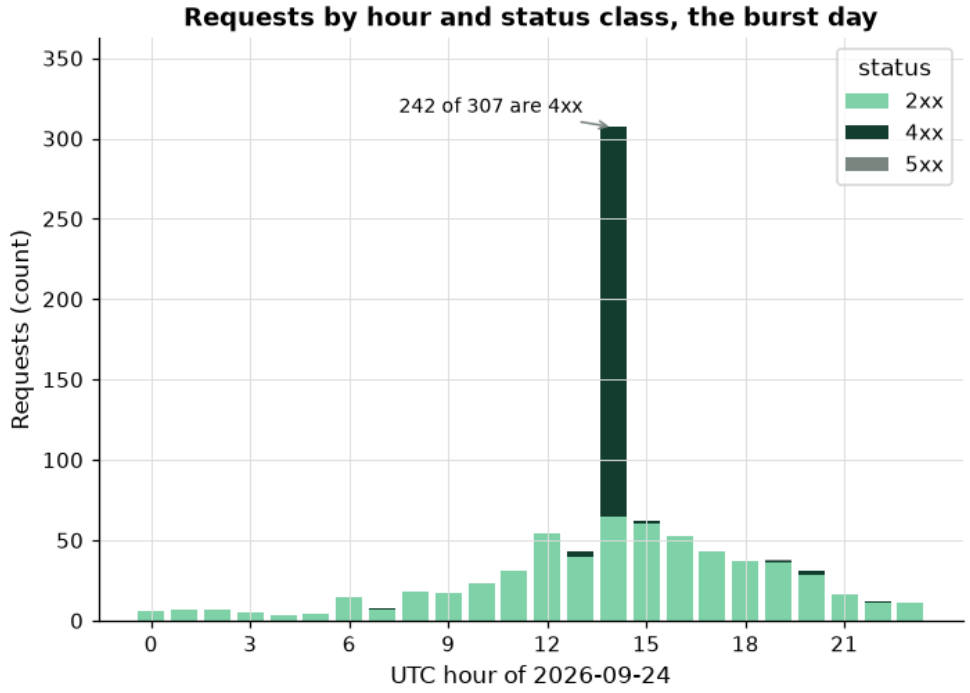
The two days, one level down

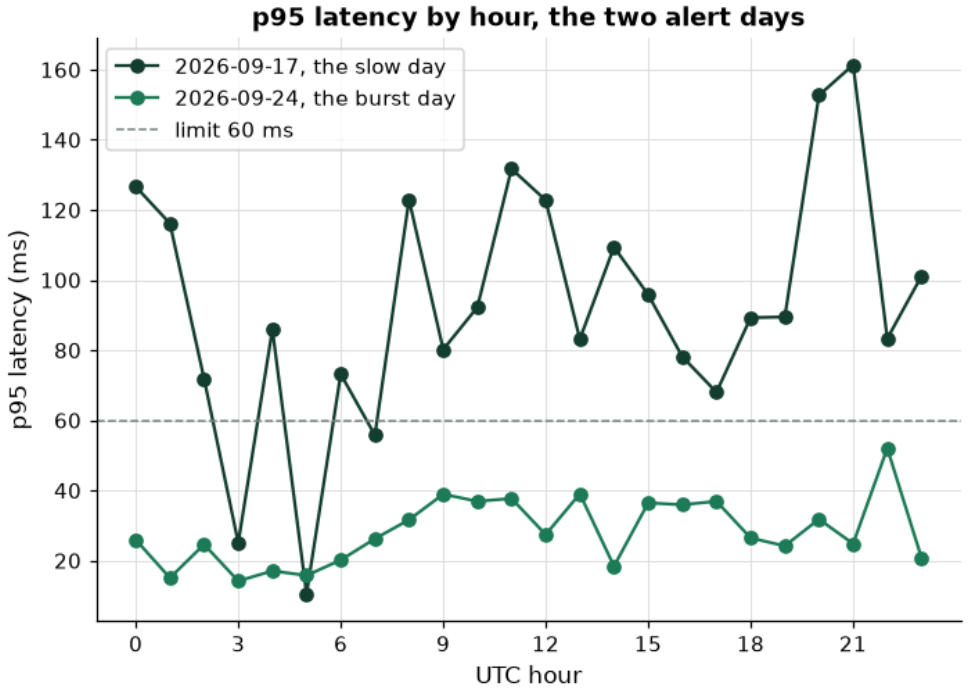




One hour carries the 422s (13:00 UTC, 1; 14:00 UTC, 240; 19:00 UTC, 1), and one day carries the slow /dv01.

One day, hour by hour





851 requests on 2026-09-24; hour 14 holds 307, 242 of them 4xx. Hours with a p95 over the limit: {'2026-09-17': 21, '2026-09-24': 0}.

Why a request log and a status page

Fastly, "Summary of June 8 outage", 8 June 2021

85% of the network returned errors; the disruption began at 09:47 UTC

"We detected the disruption within one minute"

"Within 49 minutes, 95% of our network was operating as normal."

A bug from a deployment begun May 12, "triggered by a valid customer configuration change"

Source: www.fastly.com/blog/summary-of-june-8-outage, read 2026-10-12

Seen within a minute, on one clock; and a deploy that passes its tests is not the end of watching it.

What Vercel kept of the course's requests

What the platform printed, masked

```
Recorded 2026-10-12 on the course's own account; account names, URLs, and ids masked to `student`.
> vercel logs --project pfi-bond-service --since 30m
TIME         HOST                          LEVEL
12:16:31.93  student.vercel.app  info   ? GET /health
12:16:31.68  student.vercel.app  info   ? POST /price
12:16:31.38  student.vercel.app  info   ? GET /status
```

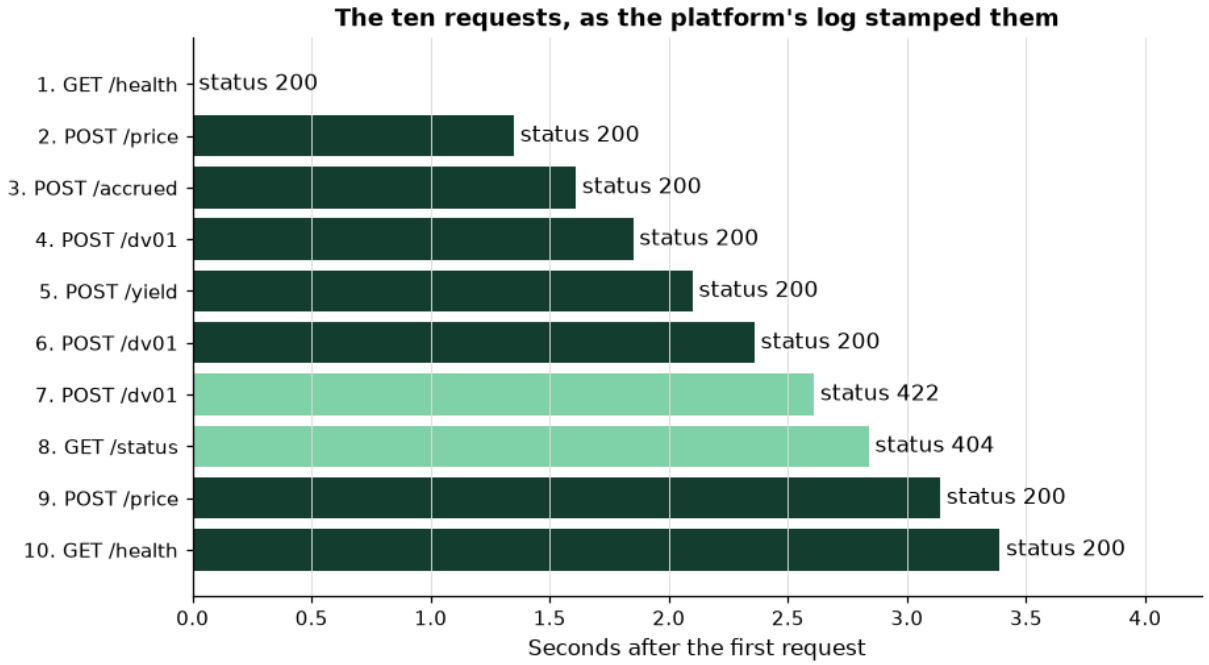
Kept with no code of yours: time, method, path, status, host, deployment, ids, the query string.

Not in the JSON: a latency or duration field. message empty: the app prints nothing.

The ten statuses on Vercel: the same as the test client's.

Hobby keeps an hour. Your own latency_ms column is what makes a p95 possible.

The ten requests, as stamped



Request	Status	Seconds
1. GET /health	200	0.00
2. POST /price	200	1.35
3. POST /accrued	200	1.61
4. POST /dv01	200	1.85
5. POST /yield	200	2.10
6. POST /dv01	200	2.36
7. POST /dv01	422	2.61
8. GET /status	404	2.84
9. POST /price	200	3.14
10. GET /health	200	3.39

The platform kept each request's status and its time. It kept no latency.

Excel and Git

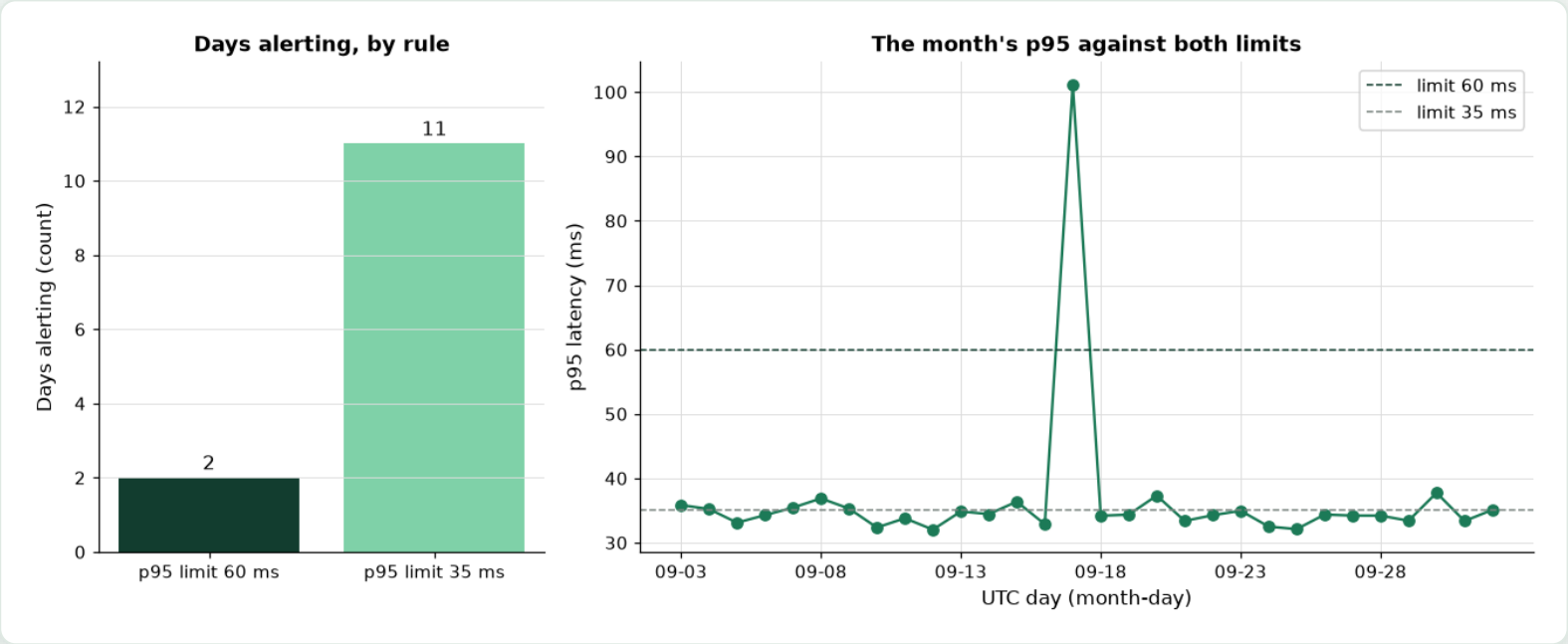
Excel side by side (tested in Excel)

`=COUNTIFS(log!C2:C19665,"/dv01")`	4,625
`=COUNTIFS(log!C2:C19665,"/price",log!D2:D19665,">=400",log!D2:D19665,"<500")`	305
`=AVERAGEIFS(log!E2:E19665,log!C2:C19665,"/dv01")`	15.2804
`=PERCENTILE.INC(FILTER(log!E2:E19665,log!C2:C19665="/dv01"),0.95)`	28.90
`=PERCENTILE.INC(hand!A2:A21,0.95) / .EXC / SMALL(...,INT(0.95*n)+1)`	19.05 / 19.95 / 20.00

```
- "p95_limit_ms": 60.0,
+ "p95_limit_ms": 35.0,
```

A p95 limit of 35 ms instead of 60: days alerting go from 2 to 11. Read the diff before you commit.

What a lower limit would mean



Rule	Days alerting
p95 limit 60 ms	2
p95 limit 35 ms	11

The ordinary days sit between the two limits: the lower one alerts on days that were fine. Read the diff first.

The AI check

The docstring is the prompt. The deploy code that comes back runs and looks finished.

1

Docstring

Units, conventions, and what the code must never let through: what you asked for.

→

2

Code back

Written for the course in the style of an AI answer. Read it.

→

3

Test first

Plant the case the docstring promises to catch. Run it: it fails.

→

4

Fix

Find the rule it broke, fix it, run the test again.

Today: sorted(x)[int(0.95 * n)] agrees on the month and returns 20.00 on 1 to 20 ms, where linear gives 19.05.

Code that runs is not the same as code that is right.

What you can do now

- Keep seven columns; never a body, query value, header, or address
- Log every request with a middleware: a file here, print on a platform
- Summarise a month by path, by percentiles
- Take a p95 the way PERCENTILE.INC does, and name the method
- Find a slow day and a burst of errors by rule

Week 3, Day 3 Model files and drift